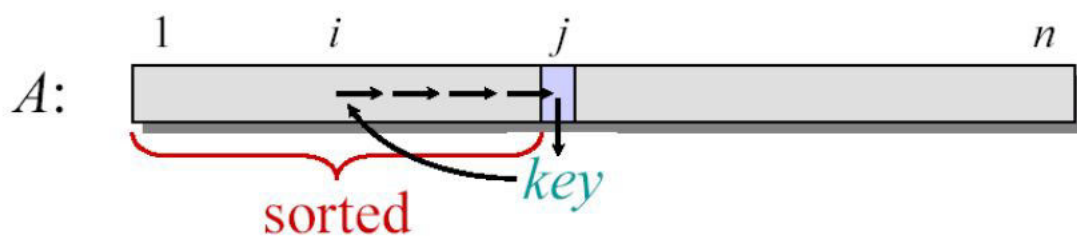


Insertion Sort

Insertion Sort

“pseudocode” {

```
INSERTION-SORT ( $A, n$ )  ▷  $A[1 \dots n]$   
  for  $j \leftarrow 2$  to  $n$   
    do  $key \leftarrow A[j]$   
       $i \leftarrow j - 1$   
      while  $i > 0$  and  $A[i] > key$   
        do  $A[i+1] \leftarrow A[i]$   
           $i \leftarrow i - 1$   
       $A[i+1] = key$ 
```



Best/Worst/Average Case

- **Best case:**

elements already sorted, $t_j=1$,
running time = $f(n)$, i.e., *linear time*.

- **Worst case:**

elements are sorted in inverse order,
 $t_j=j$, running time = $f(n^2)$, i.e., *quadratic time*

- **Average case:**

$t_j=j/2$, running time = $f(n^2)$,
i.e., *quadratic time*

Merge Sort & Recurrence Relation

Merge-Sort

MERGE-SORT(A, p, r)

```
1  if  $p < r$   
2       $q = \lfloor (p + r)/2 \rfloor$   
3      MERGE-SORT( $A, p, q$ )  
4      MERGE-SORT( $A, q + 1, r$ )  
5      MERGE( $A, p, q, r$ )
```

MERGE(A, p, q, r)

```
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

Reference: Pg#31
Introduction to algorithms

Worst-case running time $T(n)$ of Merge sort:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

- Merge sort, with its $\theta(n \lg n)$ running time, outperforms insertion sort, whose running time is $\theta(n^2)$, in the worst case.

Time Complexity (Using Master Theorem)

- **Recurrence Relation**

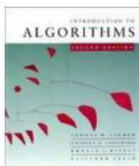
$$T(n) = 2T(n/2) + n$$

Using Master Theorem applying case 2:

$$\Theta\left(n^{\log_b a} \log n\right)$$

So time complexity is $O(n \log n)$

Quick Sort



Divide and conquer

Quicksort an n -element array:

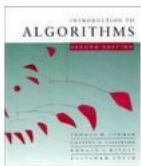
1. Divide: Partition the array into two subarrays around a **pivot** x such that elements in lower subarray $\leq x \leq$ elements in upper subarray



2. Conquer: Recursively sort the two subarrays.

3. Combine: Trivial.

Key: *Linear-time partitioning subroutine.*

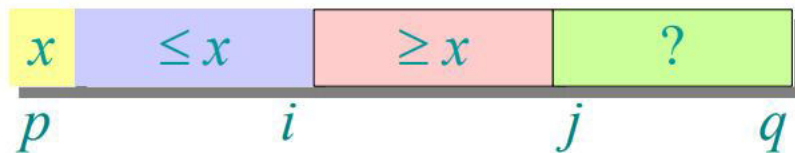


Partitioning subroutine

```
PARTITION( $A, p, q$ )  $\triangleright A[p \dots q]$   
   $x \leftarrow A[p]$   $\triangleright$  pivot =  $A[p]$   
   $i \leftarrow p$   
  for  $j \leftarrow p + 1$  to  $q$   
    do if  $A[j] \leq x$   
      then  $i \leftarrow i + 1$   
         exchange  $A[i] \leftrightarrow A[j]$   
  exchange  $A[p] \leftrightarrow A[i]$   
  return  $i$ 
```

Running time
= $O(n)$ for n
elements.

Invariant:

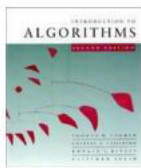


✓ Correct Randomized Partition (1-based index)

CSS

 Copy code

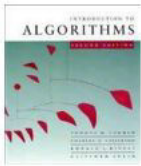
```
RANDOMIZED-PARTITION(A, p, q):  
    rand = random-in-range(p, q)  
    exchange A[p] ↔ A[rand]    ▷ move pivot to front  
  
    x = A[p]  
    i = p  
  
    for j = p+1 to q:  
        if A[j] <= x:  
            i = i + 1  
            exchange A[i] ↔ A[j]  
  
    exchange A[p] ↔ A[i]  
    return i
```



Pseudocode for quicksort

```
QUICKSORT( $A, p, r$ )  
  if  $p < r$   
  then  $q \leftarrow \text{PARTITION}(A, p, r)$   
        QUICKSORT( $A, p, q-1$ )  
        QUICKSORT( $A, q+1, r$ )
```

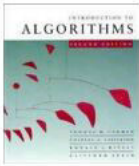
Initial call: QUICKSORT($A, 1, n$)



Worst-case of quicksort

- Input sorted or reverse sorted.
- Partition around min or max element.
- One side of partition always has no elements.

$$\begin{aligned}T(n) &= T(0) + T(n-1) + \Theta(n) \\&= \Theta(1) + T(n-1) + \Theta(n) \\&= T(n-1) + \Theta(n) \\&= \Theta(n^2) \quad \text{\textit{(arithmetic series)}}\end{aligned}$$



Best-case analysis

(For intuition only!)

If we're lucky, PARTITION splits the array evenly:

$$\begin{aligned} T(n) &= 2T(n/2) + \Theta(n) \\ &= \Theta(n \lg n) \quad (\text{same as merge sort}) \end{aligned}$$

Quicksort

- Quicksort pros [advantage]:
 - Sorts in place
 - Sorts $O(n \lg n)$ in the average case
 - Very efficient in practice , it's quick
- Quicksort cons [disadvantage]:
 - Sorts $O(n^2)$ in the worst case
 - And the worst case doesn't happen often ... sorted

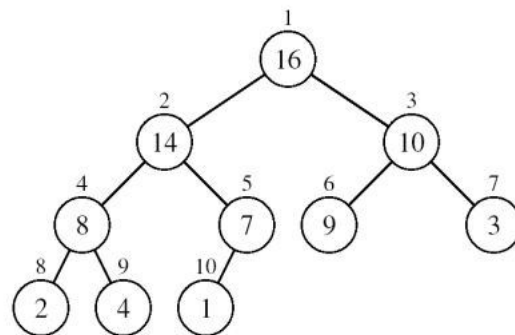
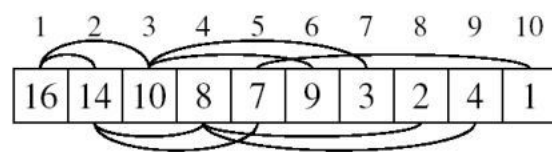
Heap Sort

Heapsort

- Running time of heapsort is $O(n \log_2 n)$
- It sorts **in place**
- It uses a data structure called a *heap*
- The heap data structure is also used to implement a priority queue efficiently

Array Representation of Heaps

- A heap can be stored as an array A .
 - Root of tree is $A[1]$
 - Left child of $A[i] = A[2i]$
 - Right child of $A[i] = A[2i + 1]$
 - Parent of $A[i] = A[\lfloor i/2 \rfloor]$
 - $\text{Heapsize}[A] \leq \text{length}[A]$
- The elements in the subarray $A[(\lfloor n/2 \rfloor + 1) \dots n]$ are leaves

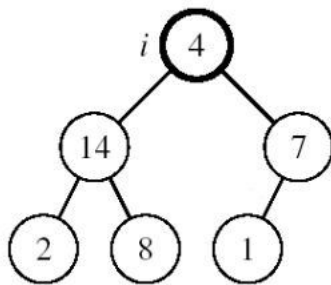


Operations on Heaps

- **Maintain/Restore the max-heap property**
 - **MAX-HEAPIFY**
- **Create a max-heap from an unordered array**
 - **BUILD-MAX-HEAP**
- **Sort an array in place**
 - **HEAPSORT**
- **Priority queues**

Maintaining the Heap Property

- Assumptions:
 - Left and Right subtrees of i are max-heaps
 - $A[i]$ may be smaller than its children



Alg: MAX-HEAPIFY(A, i, n)

1. $l \leftarrow \text{LEFT}(i)$
2. $r \leftarrow \text{RIGHT}(i)$
3. **if** $l \leq n$ and $A[l] > A[i]$
4. **then** $\text{largest} \leftarrow l$
5. **else** $\text{largest} \leftarrow i$
6. **if** $r \leq n$ and $A[r] > A[\text{largest}]$
7. **then** $\text{largest} \leftarrow r$
8. **if** $\text{largest} \neq i$
9. **then** exchange $A[i] \leftrightarrow A[\text{largest}]$
10. MAX-HEAPIFY($A, \text{largest}, n$)

MAX-HEAPIFY Running Time

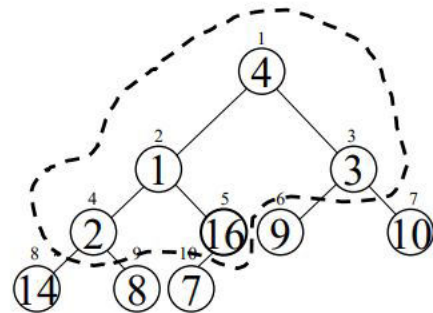
- **Intuitively:**
 - It traces a path from the root to a leaf (longest path length h)
 - At each level, it makes exactly 2 comparisons
 - Total number of comparisons is $2h$
 - Running time is $O(h)$ or $O(\lg n)$
- **Running time of MAX-HEAPIFY is $O(\lg n)$**
- **Can be written in terms of the height of the heap, as being $O(h)$**
 - Since the height of the heap is $\lfloor \lg n \rfloor$

Building a Heap

- Convert an array $A[1 \dots n]$ into a max-heap ($n = \text{length}[A]$)
- The elements in the subarray $A[(\lfloor n/2 \rfloor + 1) \dots n]$ are leaves
- Apply MAX-HEAPIFY on elements between 1 and $\lfloor n/2 \rfloor$

Alg: BUILD-MAX-HEAP(A)

1. $n = \text{length}[A]$
2. for $i \leftarrow \lfloor n/2 \rfloor$ downto 1
3. do MAX-HEAPIFY(A, i, n)



A:

4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---

Running Time of BUILD MAX HEAP

Alg: BUILD-MAX-HEAP(A)

1. $n = \text{length}[A]$
 2. **for** $i \leftarrow \lfloor n/2 \rfloor$ **downto** 1
 3. **do** MAX-HEAPIFY(A, i, n)
- $O(\lg n) \left. \vphantom{\begin{matrix} 1 \\ 2 \\ 3 \end{matrix}} \right\} O(n)$

$\Rightarrow$ Running time: $O(n \lg n)$

- **This is not an asymptotically tight upper bound**

Running time of Heapsort

HEAPSORT (A)

1	BUILD-MAX-HEAP (A)	$O(n)$
2	for $i \leftarrow \text{length}[A]$ downto 2 do	$O(n-1)$
3	exchange $A[1] \leftrightarrow A[i]$	$O(1)$
4	heap-size[A] $\leftarrow$ heap-size[A] - 1	$O(1)$
5	MAX-HEAPIFY (A, 1)	$O(\lg n)$

Total time is:

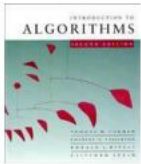
$$O(n) + O(n-1) * [O(1) + O(1) + O(\lg n)]$$

which is approximately

$$O(n) + O(n \lg n)$$

or just $O(n \lg n)$

Linear Time Sort



Sorting in linear time

Counting sort: No comparisons between elements.

- **Input:** $A[1 \dots n]$, where $A[j] \in \{1, 2, \dots, k\}$.
- **Output:** $B[1 \dots n]$, sorted.
- **Auxiliary storage:** $C[1 \dots k]$.

Counting sort assumes that each of the n input elements is an integer in the range 0 to k ,



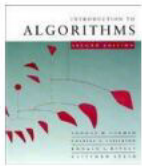
Counting sort

```
for  $i \leftarrow 1$  to  $k$ 
  do  $C[i] \leftarrow 0$ 
for  $j \leftarrow 1$  to  $n$ 
  do  $C[A[j]] \leftarrow C[A[j]] + 1$      $\triangleright C[i] = |\{\text{key} = i\}|$ 
for  $i \leftarrow 2$  to  $k$ 
  do  $C[i] \leftarrow C[i] + C[i-1]$      $\triangleright C[i] = |\{\text{key} \leq i\}|$ 
for  $j \leftarrow n$  downto  $1$ 
  do  $B[C[A[j]]] \leftarrow A[j]$ 
    $C[A[j]] \leftarrow C[A[j]] - 1$ 
```



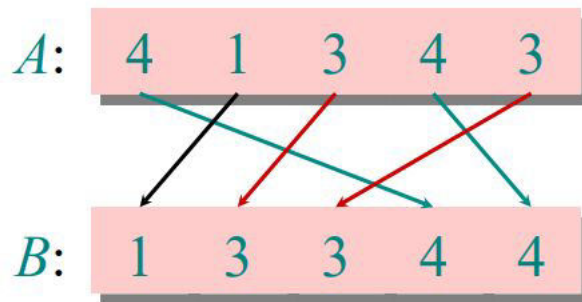
Analysis

$\Theta(k)$	{	for $i \leftarrow 1$ to k do $C[i] \leftarrow 0$
$\Theta(n)$	{	for $j \leftarrow 1$ to n do $C[A[j]] \leftarrow C[A[j]] + 1$
$\Theta(k)$	{	for $i \leftarrow 2$ to k do $C[i] \leftarrow C[i] + C[i-1]$
$\Theta(n)$	{	for $j \leftarrow n$ downto 1 do $B[C[A[j]]] \leftarrow A[j]$ $C[A[j]] \leftarrow C[A[j]] - 1$
$\Theta(n + k)$		

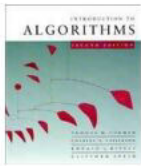


Stable sorting

Counting sort is a *stable* sort: it preserves the input order among equal elements.

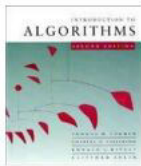


Exercise: What other sorts have this property?

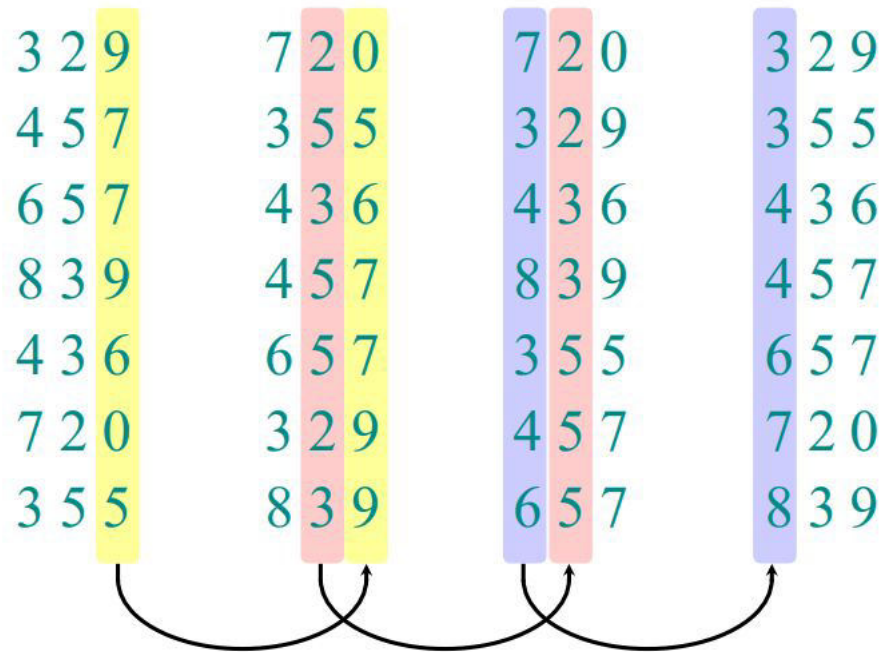


Radix sort

- **Origin:** Herman Hollerith's card-sorting machine for the 1890 U.S. Census. (See Appendix ⓘ)
- Digit-by-digit sort.
- Hollerith's original (bad) idea: sort on most-significant digit first.
- Good idea: Sort on *least-significant digit first* with auxiliary *stable* sort.



Operation of radix sort



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

✓ Full Algorithm (with helper call)

cpp

 Copy code

```
RADIX-SORT(A, d)
1. for  $i \leftarrow 1$  to  $d$  //  $i$  = current digit position (LSD to MSD)
2.    $A \leftarrow$  COUNTING-SORT-BY-DIGIT( $A, i$ )
3. return  $A$ 
```

◆ Helper: COUNTING-SORT-BY-DIGIT

This is the **digit-aware** version of counting sort.

cpp

 Copy code

```
COUNTING-SORT-BY-DIGIT(A, digitIndex)
1. let n = length[A]
2. let B[1..n] be a new array (output)
3. let C[0..9] be a count array initialized to 0 // base 10 digits (0-9)

4. // Count occurrences of each digit
5. for j ← 1 to n
6.     digit ← extractDigit(A[j], digitIndex) // e.g. for 329, digitIndex=1 → 9
7.     C[digit] ← C[digit] + 1

8. // Cumulative sum (to get positions)
9. for i ← 1 to 9
10.    C[i] ← C[i] + C[i-1]

11. // Build output (stable sort)
12. for j ← n downto 1
13.    digit ← extractDigit(A[j], digitIndex)
14.    B[C[digit]] ← A[j]
15.    C[digit] ← C[digit] - 1

16. return B
```

Radix Sort

Correctness and Running Time

- What is the running time of radix sort?

When k is not too large, use counting sort as a stable sort

- Each pass over the d digits takes time $O(n+k)$,

counting sort

- **Running Time** = Running time of stable Sort $\times d$

- $= \Theta d.(n+k)$

- K and d constants

-

- $T(n) = \Theta(n)$

- Stable, Fast

- Doesn't sort in place (because counting sort is used)

Bucket Sort

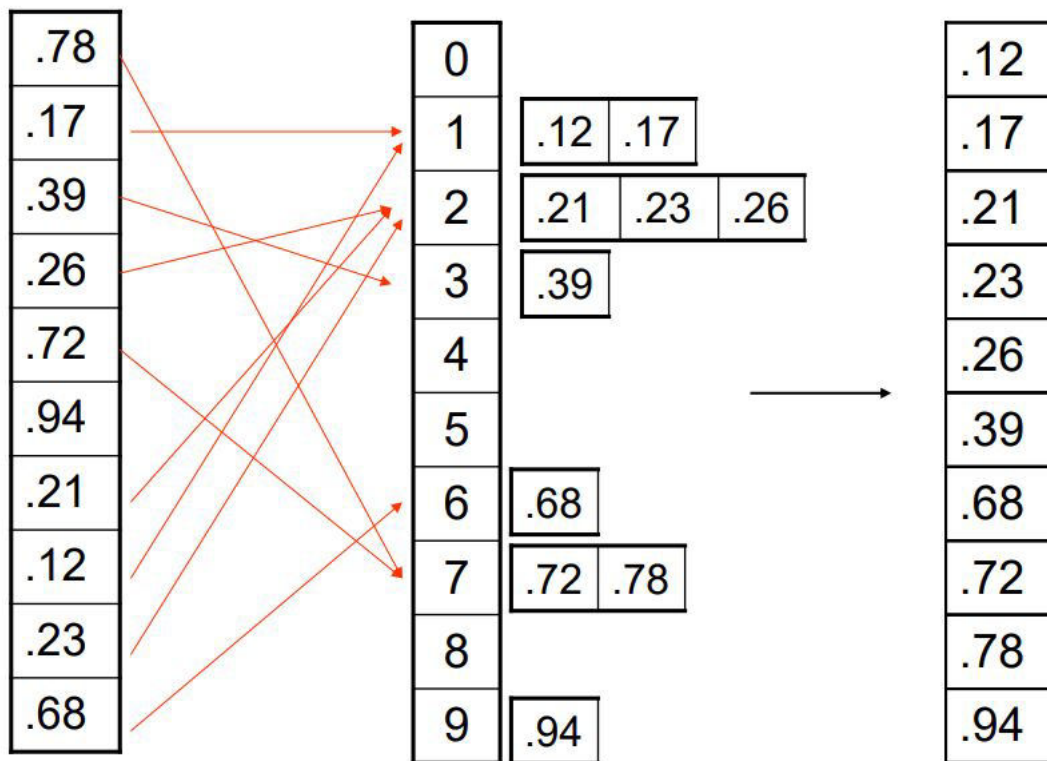
- **Assumption:** input - n real numbers from $[0, 1)$
- **Basic idea:**
 - Create n linked lists (*buckets*) to divide interval $[0, 1)$ into subintervals of size $1/n$
 - Add each input element to appropriate bucket and sort buckets with insertion sort
- **Uniform input distribution $\rightarrow O(1)$ bucket size**
 - Therefore the expected total time is $O(n)$

Bucket Sort

Bucket-Sort(A)

1. $n \leftarrow \text{length}(A)$
2. for $i \leftarrow 0$ to n  *Distribute elements over buckets*
3. do insert $A[i]$ into list $B[\text{floor}(n \cdot A[i])]$
4. for $i \leftarrow 0$ to $n - 1$
5. do Insertion-Sort($B[i]$)  *Sort each bucket*
6. Concatenate lists $B[0], B[1], \dots B[n - 1]$ in order

Bucket Sort Example



Bucket Sort – Running Time

- All lines except line 5 (Insertion-Sort) take $O(n)$ in the worst case.
- In the worst case, $O(n)$ numbers will end up in the same bucket, so in the worst case, it will take $O(n^2)$ time.
- **Lemma:** *Given that the input sequence is drawn uniformly at random from $[0,1)$, the expected size of a bucket is $O(1)$.*
- So, in the *average case*, only a constant number of elements will fall in each bucket, so it will take $O(n)$ (see proof in book).
- Use a different indexing scheme (hashing) to distribute the numbers uniformly.

Sorting Algorithms

Algorithm	Worst-case running time	Average-case/expected running time
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Heapsort	$O(n \lg n)$	—
Quicksort	$\Theta(n^2)$	$\Theta(n \lg n)$ (expected)
Counting sort	$\Theta(k + n)$	$\Theta(k + n)$
Radix sort	$\Theta(d(n + k))$	$\Theta(d(n + k))$
Bucket sort	$\Theta(n^2)$	$\Theta(n)$ (average-case)

STRING MATCHING

STRING MATCHING: INTRODUCTION

- Two Types:
 - **Exact String Matching**
 - **Approximate String Matching**

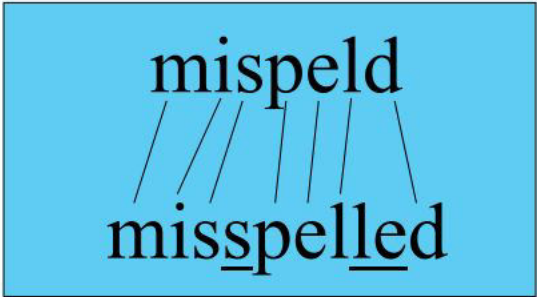


EXACT STRING MATCHING PROBLEM

- P occurs with **shift s** in T or P occurs beginning at **position $s+1$** in text T if
$$0 \leq s \leq n-m \text{ and } T[s+1..s+m] = P[1..m]$$
- i.e., $T[s+j] = P[j]$, for $1 \leq j \leq m$
- If P occurs with shift s , s is a **valid shift**; otherwise we call s an **invalid shift**.
- **String matching problem**: finding all valid shifts with which a given pattern P occurs in a given text T

APPROXIMATE STRING MATCHING

- Given a text to search (T) and a pattern to look for (P).
- Find all of the occurrences of P that exist in T, allowing a defined number of errors to be present in the matches



mispeld
misspelled

Running Time of String Matching

Algorithm	Preprocessing time
Naive	0
Rabin-Karp	$\Theta(m)$
Finite automaton	$O(m \Sigma)$
Knuth-Morris-Pratt	$\Theta(m)$

NAÏVE STRING MATCHING ALGORITHM

The naive algorithm finds all valid shifts using a loop that checks the condition $P[1..m] = T[s + 1..s + m]$ for each of the $n - m + 1$ possible values of s .

NAIVE-STRING-MATCHER(T, P)

```
1   $n = T.length$ 
2   $m = P.length$ 
3  for  $s = 0$  to  $n - m$ 
4      if  $P[1..m] == T[s + 1..s + m]$ 
5          print "Pattern occurs with shift"  $s$ 
```

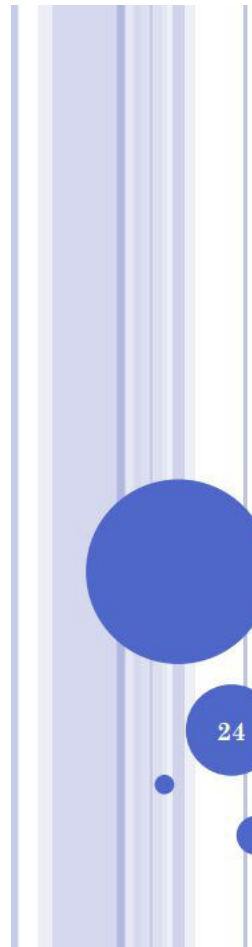
$O((n-m+1) m)$

The test in line 4 determines whether the current shift is valid; this test implicitly loops to check corresponding character positions until all positions match successfully or a mismatch is found.

NAÏVE STRING MATCHING ALGORITHM

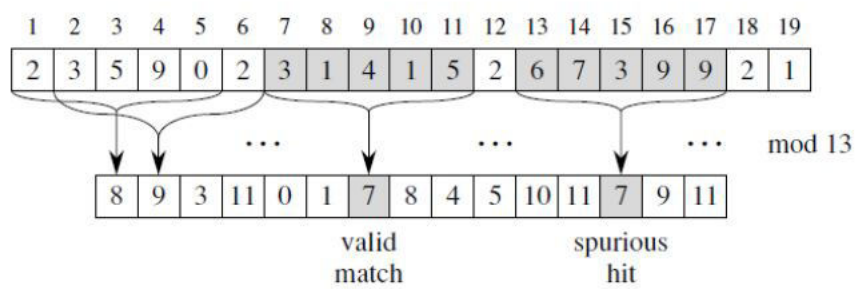
- Finds all valid shifts using a loop that tests $P[1..m] = T[s+1..s+m]$ for each of $n-m+1$ values of s
- Takes $\Theta((n-m+1)m)$ time.
- If $m=n/2$, it becomes $\Theta(n^2)$



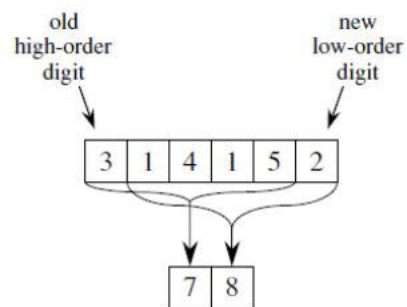


RABIN KARP ALGORITHM

24



(b)



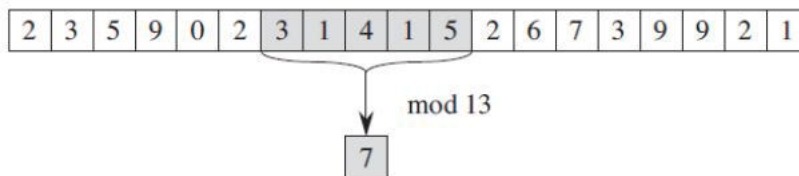
$$\begin{aligned}
 14152 &\equiv (31415 - 3 \cdot 10000) \cdot 10 + 2 \pmod{13} \\
 &\equiv (7 - 3 \cdot 3) \cdot 10 + 2 \pmod{13} \\
 &\equiv 8 \pmod{13}
 \end{aligned}$$

(c)

Fig 32.5

THE RABIN-KARP ALGORITHM (CONT'D)

- Similarly compute t_o in $\Theta(m)$ time from $T[1..m]$
- Use t_o to compute each of $t_1, t_2, \dots, t_{n-m}$ in constant time, which totals to $\Theta(n-m)$ time.
- $t_{s+1} = 10(t_s - 10^{m-1}T[s+1]) + T[s+m+1]$
- e.g., $m=5$ and $t_s=31415$ and $T[s+5+1]=2$, $s=0$
 - $t_{s+1} = 10(31415 - 10000.3) + 2$
 - $= 14152$



THE RABIN-KARP ALGORITHM (CONT'D)

- It takes $\Theta(m)$ preprocessing time and $\Theta((n-m+1)m)$ matching time in the **worst case**.
- In many applications, we expect few valid shifts (some constant c), then the running time is $O((n-m+1) + cm)$ + time for spurious hits
- and if q is large enough ($q \geq m$), we can reduce spurious hits, which gives us $O(n+m)$ running time.



Rolling Hash Concept

Instead of recalculating hash from scratch:

r

 Copy code

```
Hash(T[i+1 .. i+m]) =  
    d * (Hash(T[i .. i+m-1]) - T[i] * h) + T[i+m]
```

Where:

- d = number of characters in alphabet
- q = prime number (to reduce collisions)
- $h = d^{(m-1)} \bmod q$

Pseudocode (Exam-Ready)

r

 Copy code

```
RabinKarp(T, P, d, q):  
    n = length(T)  
    m = length(P)  
    h = d^(m-1) mod q  
  
    pHash = 0  
    tHash = 0  
  
    // Initial hash  
    for i = 0 to m-1:  
        pHash = (d * pHash + P[i]) mod q  
        tHash = (d * tHash + T[i]) mod q  
  
    for i = 0 to n - m:  
        if pHash == tHash:  
            if T[i .. i+m-1] == P:  
                report "Pattern found at index i"  
  
        if i < n - m:  
            tHash = (d * (tHash - T[i] * h) + T[i + m]) mod q  
            if tHash < 0:  
                tHash = tHash + q
```

COMPUTE-TRANSITION-FUNCTION(P, Σ, m)

1 for $q = 0$ to m

2 for each character $a \in \Sigma$

3 $k = \min \{m, q + 1\}$

4 while $P[:k]$ is not a suffix of $P[:q]a$

5 $k = k - 1$

6 $\delta(q, a) = k$

7 return δ

FINITE-AUTOMATON-MATCHER(T, δ, m)

```
1   $n \leftarrow \text{length}[T]$ 
2   $q \leftarrow 0$ 
3  for  $i \leftarrow 1$  to  $n$ 
4      do  $q \leftarrow \delta(q, T[i])$ 
5          if  $q = m$ 
6              then print “Pattern occurs with shift”  $i - m$ 
```

KMP String Matching

COMPUTE-PREFIX-FUNCTION(P)

```
1   $m \leftarrow \text{length}[P]$ 
2   $\pi[1] \leftarrow 0$ 
3   $k \leftarrow 0$ 
4  for  $q \leftarrow 2$  to  $m$ 
5      do while  $k > 0$  and  $P[k + 1] \neq P[q]$ 
6          do  $k \leftarrow \pi[k]$ 
7          if  $P[k + 1] = P[q]$ 
8              then  $k \leftarrow k + 1$ 
9           $\pi[q] \leftarrow k$ 
10 return  $\pi$ 
```

KMP-MATCHER(T, P)

```
1   $n = T.length$ 
2   $m = P.length$ 
3   $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P)$ 
4   $q = 0$  // number of characters matched
5  for  $i = 1$  to  $n$  // scan the text from left to right
6      while  $q > 0$  and  $P[q + 1] \neq T[i]$ 
7           $q = \pi[q]$  // next character does not match
8      if  $P[q + 1] == T[i]$ 
9           $q = q + 1$  // next character matches
10     if  $q == m$  // is all of  $P$  matched?
11         print "Pattern occurs with shift"  $i - m$ 
12          $q = \pi[q]$  // look for the next match
```